

INTERNSHIP REPORT

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

Submitted By

Aditi

35515002722

CSE-1 (8th Semester)

To

Mr. Sushil Kumar



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MAHARAJA SURAJMAL INSTITUTE OF TECHNOLOGY,

**(AFFILIATED TO GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY,
DELHI) JANAK PURI, NEW DELHI – 110058**

2022-26

TABLE OF CONTENTS

S. No.	Section	Page No.
1	About the organization	3-4
2	Project Introduction	5-6
3	Role and Work Assigned in Project	7-8
4	Progress and Work Completed Till presentation	9-11
5	Work Yet to Complete	11-12
6	References	13

About The Organization

Overview

Cognizant is a leading global professional services company that transforms clients' business, operating, and technology models for the digital era [1]. Headquartered in Teaneck, New Jersey, it is a Fortune 500 company with annual revenues of approximately \$19.4 Billion (2024) and a massive global workforce of over 340,000 employees.

Key Strengths

Cognizant has built a reputation for high-touch client relationships and a robust global delivery model:

- **Global Footprint:** Operations across 35+ countries with major delivery hubs in India, North America, and Europe.
- **Digital Leadership:** Recognized as a leader in AI, Cloud, and Data analytics by top analyst firms (Gartner, IDC).
- **Scale:** One of the world's largest recruiters of STEM talent.
- **Client Base:** Partners with 30 of the top 30 global pharma companies and 9 of the top 10 European banks.

Industry Presence

Cognizant operates across four main segments, providing end-to-end digital transformation:

1. **Financial Services:** Banking, Capital Markets, and Insurance.
2. **Health Sciences:** Pharmaceutical, Biotech, and Healthcare providers.
3. **Products and Resources:** Retail, Consumer Goods, Manufacturing, Logistics, Energy, and Utilities.
4. **Communications, Media, and Technology:** Entertainment, Information Services, and Software.

Product & Service Portfolio

Unlike HCL Healthcare's specific wellness apps, Cognizant's "products" are often massive enterprise platforms or specialized digital suites.

1. TriZetto Healthcare Products

Cognizant's flagship software suite for the healthcare industry [1]. It manages the business of healthcare, from member enrolment to claims processing.

- **Facets & QNXT:** Core administrative platforms that help health plans manage complex benefits and provider networks.
- **Clinical Care Management:** Tools that help insurers and providers collaborate to improve patient outcomes.

2. Cognizant Neuro® (AI & Automation)

A suite of AI-led platforms designed to orchestrate and automate business processes.

- **Neuro IT Operations:** Uses AIOps to predict and fix IT issues before they impact the business.
- **Neuro Business Processes:** Automates repetitive tasks in finance, HR, and supply chain using generative AI.

3. Ocean (Digital Engineering)

A platform approach to modernizing legacy systems.

- **Cloud Acceleration:** Tools to migrate massive enterprise data centres to AWS, Azure, or Google Cloud.

Project Introduction: TaxEase

TaxEase is a comprehensive, web-based ecosystem designed to bridge the gap between financial compliance and administrative governance. In the modern era of digital finance, taxation boards and government agencies face significant challenges in maintaining transparency, ensuring data integrity, and tracking filing outcomes across diverse taxpayer demographics.

TaxEase serves as a centralized platform for managing digital tax submissions, e-filing resources, and regulatory compliance. Unlike standard accounting software that focuses solely on the user-accountant interaction, TaxEase incorporates a robust **Governance Layer**. This layer allows government auditors and tax officers to monitor filing metrics and ensure that financial delivery aligns with national fiscal policies.

1. Project Objectives

The primary objectives of the TaxEase project include:

- **Centralized Management:** Providing a single "Source of Truth" for taxpayer records, financial documents, and filing history.
- **Accountability & Transparency:** Utilizing detailed **Audit Trails** and compliance modules to track every action within the system, ensuring high-level financial governance.
- **Scalability:** Designing a stateless architecture capable of supporting high concurrent user traffic during peak tax seasons.
- **Data-Driven Insights:** Empowering stakeholders with real-time analytics to identify gaps in tax collection and optimize resource allocation.

2. System Stakeholders (Actors)

The system is designed with a multi-tenant architecture, catering to distinct user roles, each with specific permissions and workflows:

- **Taxpayer:** The primary user. They engage in secure registration, upload financial documents, submit filings, and monitor their compliance status through personal trackers.
- **Tax Officer:** The evaluators. They manage the filing lifecycle, review digital assets (PDFs, PAN data), and approve or reject submissions based on evidence.

- **Auditor:** The external oversight. They review high-level audit logs and monitor how the system is utilized to ensure the efficient use of institutional funds and prevent fraud.
- **System Administrator:** The operational backbone. They oversee user role assignments, manage system configurations, and ensure the platform is up to date with the latest tax laws.

3. Technical Architecture & Design

The architecture of TaxEase follows a modern, scalable approach suitable for high-traffic enterprise applications.

3.1 Tech Stack

- **Frontend:** Built using **React**. Selected for its ability to handle complex, state-driven dashboards and provide a responsive experience across devices.
- **Backend:** Developed using **REST API-based Microservices (Java Spring Boot)**. This allows each module (e.g., Identity Access Management, Tax Filing, Audit) to scale independently based on demand.
- **Database:** A **Relational Database (MySQL)** is used to maintain strict data integrity, which is vital for financial records. Complex relationships between taxpayers, filings, and payments are managed via optimized SQL schemas and **JPA Cascading**.

3.2 Deployment & Security

The system is designed for **Cloud Deployment** but maintains strict data sovereignty. Security is enforced through:

- **JWT (JSON Web Tokens):** For stateless authentication, ensuring that user sessions are secure without taxing server memory.
- **Role-Based Access Control (RBAC):** Restricting access to sensitive endpoints (like Audit Logs) to only authorized personnel.
- **BCrypt Encryption:** Protecting taxpayer credentials by ensuring no plain-text passwords are ever stored in the database.

Role and Work Assigned in Project

1. Role Overview

During my internship, I served as a **Backend Developer** for the core functional engine of the TaxEase platform. My primary responsibility was to design, develop, and integrate the foundational layer of the system: the **Tax Filing and Document Management Module (Module 4.3)**

As TaxEase is a governance-focused platform involving sensitive financial stakeholders—ranging from individual Taxpayers to Government Auditors—the security, traceability, and integrity of data were paramount. My role involved ensuring the system followed a **Stateless Security Architecture**, where every financial submission is authenticated, validated, and recorded based on strictly defined regulatory roles.

2. Work Assigned

My assignment was strategically divided into three major phases to ensure a robust and scalable delivery:

Phase I: Data Architecture & Schema Design

- **Relational Mapping:** Designing a scalable database schema for tax management using **MySQL**, specifically focusing on the **TaxFiling** root entity.
- **Integrity Logic:** Defining complex relationships between Filings, Payments, and Documents using **JPA Cascading (CascadeType.ALL)** to ensure full data synchronization.
- **API Contract:** Mapping out the RESTful API endpoints required for secure document upload, filing submission, and historical data retrieval.

Phase II: Core Security & Access Control

- **Identity Management:** Implementing secure authentication mechanisms using **Java Spring Boot** and **Spring Security** to protect sensitive financial records.

- **Role-Based Access Control (RBAC):** Developing a permission-based system to manage the distinct actor types (Taxpayer, Tax Officer, and Auditor), ensuring data isolation between users.
- **Stateless Session Management:** Integrating **JWT (JSON Web Tokens)** to handle high-concurrency traffic during peak filing seasons without compromising server performance.

Phase III: Audit & Compliance Logic

- **Traceability Engine:** Developing an automated **Audit Logging System** that captures every sensitive action within the filing lifecycle (Submissions, Approvals, Rejections).
- **Audit Interceptors:** Creating service-level logic to log the "Who, What, When, and Where" for every API call, satisfying the stringent non-repudiation requirements of government financial auditing.
- **Data Validation:** Implementing **Jakarta Bean Validation** to enforce "Clean Data" entry, preventing common filing errors such as incorrect PAN formats or negative tax values.

Progress and Work Completed Till Presentation

1. Secure Authentication Engine

I successfully implemented the foundational security layer using **Spring Security**. To ensure the highest level of financial data protection, I utilized **BCrypt password hashing** for storing all user credentials within the MySQL database, ensuring that no plain-text passwords exist in the system.

- **JWT Integration:** I developed a custom **JWT (JSON Web Token)** utility class to generate, validate, and parse secure tokens. This allowed the React frontend to maintain a stateless, secure connection with the Spring Boot backend services.
- **Token Lifecycle:** I managed the token expiration logic (set to **1,800,000ms**) and implemented the backend logic to support secure session termination, ensuring that the token is invalidated upon client-side logout.

2. Role-Based Access Control (RBAC)

This was a critical component of my work to ensure the governance of tax data. Given the sensitive nature of **TaxEase** users, I implemented a granular authorization layer. I used **Spring Security Expressions** (e.g., `@PreAuthorize`) to protect sensitive API endpoints.

- **Taxpayer Role:** Access is strictly limited to personal profile management and individual filing submissions.
- **Tax Officer Role:** Extended permissions to view the filing queue, review uploaded documents, and update filing statuses (Approved/Rejected).
- **Auditor Role:** Granted "Read-Only" access to system-wide reports and **Audit Logs**, ensuring they can monitor the integrity of the tax process without the ability to alter financial data.

3. The Audit Logging Infrastructure

To satisfy the "Governance" and "Compliance" requirements of the project, I built a centralized **AuditLog** entity. Every time a user (especially a Tax Officer or Auditor) performs a sensitive action—such as approving a tax return or accessing a taxpayer's personal documents—a record is instantly created in the `audit_log` table.

- **Implementation:** I utilized **Service-Layer Interceptors** to capture metadata. This ensured that the logging logic remained decoupled from the core business logic, resulting in a cleaner and more maintainable codebase.
- **Traceability:** The system logs the "Who, What, When, and Where" for every critical transaction, providing a permanent "Source of Truth" for government oversight.

4. Database & Entity Management

I was responsible for the creation and optimization of the core schema for Module 4.3:

- **Tax Filing Table:** Designed the root entity to handle financial data, assessment years, and filing statuses.
- **Payment & Document Tables:** Developed the relational logic for document uploads where the DocumentURI points to a secure storage location. I utilized **JPA Cascading (CascadeType.ALL)** to ensure that all sub-records are correctly linked and synchronized with the parent filing.

5. Challenges Overcome

5.1 Handling Complex Permission Logic

- **Challenge:** Initially, managing the permissions for multiple roles led to complex and redundant "if-else" blocks within the controller layer.
- **Solution:** I transitioned to a **Declarative Security Model** using Spring Security annotations. By defining custom "Authorities" for each role, I reduced code redundancy and ensured the system is scalable for future administrative roles.

5.2 Preventing Unauthorized Data Access (IDOR)

- **Challenge:** Ensuring that a Taxpayer could only see their own filings and that one user could not "guess" another user's Filing ID to access private data.
- **Solution:** I implemented a **Resource Ownership Check** in the backend service. Before serving any data, the system verifies if the UserID from the JWT token matches the OwnerID of the requested tax record in the database.

Work Yet to Complete

1. Frontend Development (Angular Integration)

While the core RESTful API layer for **Module 4.3** is robust, the next major milestone is the development of the **TaxEase Responsive Dashboard** using **Angular**. This involves building a Type-Safe Single Page Application (SPA) to serve the six distinct actor types defined in the project scope.

- **Role-Specific Portals:** Developing the **Taxpayer Portal** (for filing and payment tracking) and the **Tax Officer/Auditor Dashboards** (for validation and compliance oversight) using Angular components.
- **State Management & Interceptors:** Implementing **Angular Services** to manage user sessions. We will utilize **HTTP Interceptors** to automatically attach the JWT obtained from the Identity module to the headers of every outgoing API request.
- **Reactive Forms with Validation:** Building dynamic forms for tax filing and taxpayer registration using **Angular Reactive Forms** to ensure real-time client-side validation (e.g., PAN/Regex checks) before data is sent to the Spring Boot backend.

2. Transition to Advanced Microservices Architecture

The current implementation focuses on the foundational backend logic. To meet the non-functional requirement of supporting **300,000 concurrent users**, the system must be decoupled into independent, scalable microservices.

- **Service Decomposition:** Breaking the monolith into specialized services as per the Module Overview:
 - **Filing & Document Service:** A dedicated service to handle heavy PDF/Document uploads and status transitions.
 - **Payment & Revenue Service:** A high-concurrency engine integrated with external payment gateways for real-time revenue tracking.
 - **Notification Service:** An **asynchronous service** using a message broker (like RabbitMQ) to handle SMS and Email alerts without blocking the main filing workflow.
- **Inter-Service Communication:** Implementing **Feign Clients** for synchronous communication and **Service Discovery (Eureka)** to manage the microservices network effectively.

3. Infrastructure, Security & Observability

The deployment strategy must move from a local environment to a **Production-Grade Cloud Infrastructure** to ensure the 99.9% availability target.

- **API Gateway & Load Balancing:** Implementing **Spring Cloud Gateway** as a single entry point for the Angular frontend. This will handle rate limiting and load balancing across multiple instances of the Filing and Payment services.
- **Security Hardening:** Deploying a **Web Application Firewall (WAF)** to protect against SQL Injection and XSS, and ensuring all financial data in transit is encrypted via **TLS/SSL**.
- **Centralized Logging & KPIs:** Integrating an observability stack (like ELK or Prometheus/Grafana) to monitor core performance metrics such as **Filing Completion Rate** and **Payment Success Rate**, as specified in the non-functional requirements.

References

- [1] Cognizant Technological Solutions: About Us. Retrieved from <https://www.cognizant.com/us/en/about-cognizant>